

Args and Kwargs

Think of `*args` and `**kwargs` as the "wildcards" of Python function parameters.

In Python, we use these special syntaxes when we aren't sure how many arguments might be passed to a function, or when we want to write wrappers that pass arguments through to another function without knowing what they are.

- `*args`: Allows you to pass a variable number of **positional** arguments.
- `**kwargs`: **Allows you to pass a variable number of** keyword** (named) arguments.

Note: The names `args` and `kwargs` are just strong conventions among developers.

The magic actually happens in the asterisks (`*` and `**`).

You *could* write `*var` or `**options`, but sticking to standard naming makes your code readable to other devs.

1. Understanding `*args`

The single asterisk (`*`) takes all the **extra** positional arguments passed to the function and packs them into a **tuple**.

Example: A Flexible Calculator

Imagine you want a function that sums up numbers, but you don't want to limit it to just two or three numbers.

```
def sum_all(*args):
    # args is treated as a tuple inside the function
    print(f"Type of args: {type(args)}")
    print(f"Content of args: {args}")

    total = 0
    for num in args:
        total += num
    return total

# Calling the function with different amounts of arguments
print(sum_all(1, 2))
print("----")
print(sum_all(10, 20, 30, 40, 50))
```

Output:

```
Type of args: <class 'tuple'>
Content of args: (1, 2)
3
---
Type of args: <class 'tuple'>
Content of args: (10, 20, 30, 40, 50)
150
```

2. Understanding `**kwargs`

The double asterisk (`**`) takes all the **extra** keyword arguments (arguments passed as `key=value`) and packs them into a **dictionary**.

Example: Building a User Profile

You might require a name, but allow users to add any other details they want (age, email, job title, etc.).

```
def create_profile(name, **kwargs):
    # kwargs is treated as a dictionary inside the function
    print(f"User: {name}")

    # We can iterate over the dictionary items
    for key, value in kwargs.items():
        print(f" - {key}: {value}")

# Calling with different keyword arguments
create_profile("Alice", age=30, job="Developer")
print("----")
create_profile("Bob", location="NYC", status="Active", hobbies="Chess")
```

Output:

```
User: Alice
  - age: 30
  - job: Developer
----
User: Bob
  - location: NYC
```

- status: Active
- hobbies: Chess

3. Combining `*args` and `**kwargs`

You can use both in the same function. However, **order matters**. Python requires arguments to be declared in this specific order:

1. Standard (Formal) arguments
2. `*args`
3. `**kwargs`

```
def mixed_function(arg1, *args, **kwargs):  
    print(f"First arg: {arg1}")  
    print(f"Args (tuple): {args}")  
    print(f"Kwargs (dict): {kwargs}")  
  
mixed_function('Hello', 1, 2, 3, color='red', size='large')
```

4. The Unpacking Operator (Calling functions)

This is where it gets really cool. You can use the `*` and `**` syntax not just when **defining** a function, but when **calling** one. This is called "unpacking."

If you have a list or a dictionary and you want to pass its contents as separate arguments to a function, you use these operators.

```
def specific_func(a, b, c):  
    print(f"a={a}, b={b}, c={c}")  
  
my_list = [1, 2, 3]  
my_dict = {'a': 10, 'b': 20, 'c': 30}  
  
# Unpacking list into positional arguments  
# Equivalent to specific_func(1, 2, 3)  
specific_func(*my_list)  
  
# Unpacking dictionary into keyword arguments
```

```
# Equivalent to specific_func(a=10, b=20, c=30)
specific_func(**my_dict)
```

Summary Table

Syntax	Name	Data Type Inside Function	Usage
<code>*args</code>	Positional Args	<code>Tuple</code>	Passing optional, non-named arguments.
<code>**kwargs</code>	Keyword Args	<code>Dictionary</code>	Passing optional, named arguments (<code>key=value</code>).